Архитектура компьютера
и операционные системы

Отчёт по индивидуальному заданию 2

Вариант 20
Автогонки

Многопоточная консольная программа
на C для Linux
Версия 2.0

Выполнил
Куслин Глеб Витальевич
Группа БПИ252

2026

1 Условие задачи и принятая модель

В работе реализована многопоточная модель автогонок. Главный поток выполняет роль судьи, каждой машине соответствует отдельный поток pthread. Судья определяет порядок и бюджет ходов, водители выбирают и выполняют маршруты. Программа показывает движение по клеткам, сохраняет журнал и формирует результаты машин и команд. Взаимодействие организовано с помощью мьютекса и семафоров.

Условие варианта 20 приведено в i-tasks.pdf на страницах 40–42. Требования к ИДЗ 2 взяты из i-tasks-req-02.pdf, страницы 2 и 5–7. По сравнению с первой версией сохранены правила гонки, но ходы перенесены в самостоятельные потоки машин.

Правила гонки

Базовая команда состоит из двух машин. Команды участвуют в жеребьёвке, затем машины размещаются на внешней дорожке перед линией старта: сначала первые машины всех команд, затем следующие. Число команд и автомобилей можно изменить при запуске.

В начале раунда активные машины упорядочиваются по продвижению вперёд. При равенстве преимущество имеет внутренняя дорожка; окончательное равенство разрешается по номеру машины. Очередь сохраняется до конца раунда.

Лидер получает бюджет 4, что соответствует разрешённому выбору от 1 до 4. Вторая машина получает фактическую стоимость предыдущего хода плюс 2, остальные — плюс 1. Переход вперёд стоит 1, внутрь — 1, наружу — 3. Запрещены движение назад, диагонали, занятые клетки и повторное посещение любой клетки маршрута, включая исходную.

Если допустимого первого перехода нет, ход нулевой. Такая машина сходит после завершения всего раунда. Короткий непустой ход не считается нулевым. Финишировавшие машины также остаются препятствиями до конца раунда, после чего снимаются с трассы.

Геометрия и классификация

Трасса представлена кольцом: каждая дорожка содержит одинаковое число L клеток, последняя клетка соединена с первой. Дорожка 1 — внутренняя; боковой переход сохраняет продольную координату. Это упрощённая клеточная модель: точная геометрия поворотов рисунка, переменная ширина и разные длины дорожек не воспроизводятся.

Общая линия старта и финиша находится в клетке 1. У первой стартующей машины продвижение равно −1, у следующих −2 и далее. Первое пересечение линии даёт продвижение 0; круг заканчивается при L, 2L и далее. Финиш наступает точно при K·L, где K — число кругов. Боковой переход расходует бюджет, но не увеличивает продвижение.

Финишировавшие ранжируются по очереди финиша, остальные — по продвижению, внутренней дорожке и номеру. Место команды определяется её лучшей машиной. Круговые премии и дополнительное снятие отставших отключены; это явно сообщается при запуске.

2 Данные и алгоритм движения

Состояние модели отделено от взаимодействия потоков и отображения. Это позволяет проверять правила движения независимо от способа выдачи ходов.

Модуль	Назначение
main.c	Аргументы, журнал, сигналы, запуск и остановка, итоговый код возврата
race.c и race.h	Модель трассы, маршруты, раунды и результаты
threads.c и threads.h	Потоки машин, задания, подтверждения и освобождение ресурсов
display.c и display.h	Текстовый вывод, журнал, терминальные кадры и задержки
tests/	Проверки модели, потоков, журналов, кадров и ошибок

Config хранит параметры запуска. Car содержит номер, команду, дорожку, клетку, продвижение, состояние, признак нулевого хода и место финиша. Route содержит последовательность направлений, число переходов и суммарную стоимость. Race объединяет конфигурацию, массив машин, занятость трассы, очередь, число раундов и финишей.

В массиве track ноль означает свободную клетку, другое значение — номер машины. Координаты Car и track обновляются при каждом переходе. Состояния ACTIVE, FINISHED, RETIRED и UNFINISHED различают участие в гонке, финиш, сход и остановку без завершения дистанции.

Выбор и выполнение маршрута

choose_route строит путь до изменения состояния гонки. Для каждого шага проверяются границы дорожек, занятость, остаток бюджета и множество посещённых клеток. Исходная клетка сразу отмечается посещённой, поэтому обход короткого кольца не позволяет вернуться в неё за один ход.

Стратегия greedy выбирает первый допустимый переход в порядке «вперёд, внутрь, наружу». random случайно выбирает один из допустимых переходов. Движение продолжается до исчерпания бюджета, отсутствия продолжения или финиша. Стратегии локальные: полный поиск наилучшего маршрута не выполняется.

execute_route последовательно освобождает прежнюю клетку и занимает новую, увеличивает продвижение только при движении вперёд, записывает событие и показывает кадр. При достижении финиша маршрут обрывается точно на линии. perform_turn возвращает фактическую стоимость, по которой судья рассчитывает следующий бюджет.

Границы данных

Поддерживаются до 32 машин, 6 дорожек и 120 клеток на дорожке. Для стартовой колонны необходимо L > N, где N — число машин. Максимальный бюджет последней из 32 машин равен 4 + 2 + 30 = 36. Каждый переход стоит не меньше 1, поэтому массив из MAX_CARS + 4 направлений достаточен для любого допустимого хода.

При максимальных 100 кругах продвижение не превышает 12000; начальное значение не меньше −32. Используются фиксированные массивы. Параметры проверяются до создания гонки и потоков.

3 Потоки и синхронизация

В одном процессе работают N потоков машин и главный поток судьи. По умолчанию N = 6 и всего потоков 7; максимум — 33. Потоки создаются один раз на всю гонку. После схода или финиша поток машины продолжает ждать общей остановки, но больше не получает игровых заданий.

Судья и водители являются активными сущностями варианта. Команды хранят принадлежность и участвуют в классификации, трасса хранит клетки; самостоятельных действий у них в принятой модели нет. Дополнительные потоки для них не создаются.

Объекты взаимодействия

Объект	Количество	Назначение
mutex	1	Защита Race, заданий, результатов и признака остановки
permission	N	Разрешение хода конкретной машине
completed	1	Подтверждение окончания хода судье

Семафоры безымянные, общие для потоков одного процесса; их начальные значения равны 0. CarWorker хранит pthread машины, её индекс, permission, переданный бюджет limit и возвращённую стоимость cost. RaceThreads хранит mutex, completed, массив работников и признаки созданных ресурсов.

Передача одного хода

1. Судья под mutex начинает раунд и фиксирует очередь активных машин.
2. Для очередной машины под mutex записывает limit, освобождает mutex и выполняет sem_post(permission).
3. Машина выходит из sem_wait(permission), захватывает mutex и проверяет stopping. При обычной работе выполняет perform_turn, записывает cost и освобождает mutex.
4. Машина выполняет sem_post(completed), затем снова ждёт permission.
5. Судья завершает sem_wait(completed), под mutex читает cost и рассчитывает бюджет следующего участника.
6. После последнего подтверждения судья под mutex снимает выбывших и формирует очередь следующего раунда.

Весь ход, включая выбор пути, изменение модели, журнал, кадры и паузы, выполняется под одним mutex. Это соответствует правилам с последовательными ходами. Одновременно движется одна машина: следующий бюджет зависит от предыдущего результата. Независимые работники получают разрешения и исполняют ходы в собственных pthread.

Порядок ожиданий

Судья не удерживает mutex при sem_wait(completed) или pthread_join. Машины ждут permission без mutex. Ожидание завершения хода не блокирует доступ исполнителя к состоянию. Новое задание выдаётся после подтверждения предыдущего, поэтому один completed не смешивает результаты.

Семафор сохраняет разрешение, даже если sem_post произошёл раньше sem_wait. При EINTR ожидание повторяется. Нулевой ход также подтверждается один раз. Генератор rand вызывается только текущим работником под mutex; его инициализация выполнена до запуска работников.

4 Запуск и завершение потоков

После проверки аргументов программа открывает журнал, устанавливает обработчики сигналов, создаёт модель и вызывает start_workers. Флаги и счётчики отмечают каждый успешно созданный mutex, семафор и pthread. Race и данные работников существуют до присоединения всех потоков.

Штатное завершение

Главный цикл выполняет целые раунды, пока есть активные машины, не достигнут предел раундов и не получен сигнал. Если все машины финишировали или сошли, формируются окончательные результаты. При остановке по пределу оставшиеся ACTIVE получают состояние UNFINISHED.

После последнего раунда незавершённых заданий нет. stop_workers устанавливает stopping, освобождает mutex и отправляет permission каждому созданному работнику. Проснувшийся поток видит stopping и возвращается без нового хода. Судья вызывает pthread_join для всех работников, затем уничтожает семафоры и mutex.

После присоединения потоков восстанавливаются экран и курсор, печатаются итоги, проверяются ошибки stdout и журнала, журнал закрывается. Работники не обращаются к закрытому файлу или уничтоженному объекту синхронизации.

Сигналы

SIGINT и SIGTERM устанавливают атомарный флаг interrupted. Обработчик не печатает, не закрывает файлы и не вызывает функции синхронизации. Требование lock-free atomic_int проверяется при компиляции. sigaction использует SA_RESTART, sem_wait дополнительно повторяется при EINTR.

При сигнале новый раунд не начинается, уже начатый завершается целиком. Паузы отображения разбиты на части не длиннее 20 мс и отменяются после сигнала. Большой --delay не заставляет ждать все паузы до конца раунда. Если сигнал пришёл во время создания работников, создание заканчивается, игровые раунды не запускаются, затем выполняется остановка.

Частичный запуск и ошибки

При ошибке инициализации или pthread_create выполняется общий откат: пробуждаются и присоединяются только созданные работники, освобождаются только инициализированные примитивы. Ошибка первого mutex_init не приводит к обращению к несуществующему mutex.

После join ошибка одного destroy не прерывает обработку остальных объектов. Сохраняется первая ошибка; флаг ресурса сбрасывается только после успешного destroy. Повторный вызов после успешной остановки безопасен. При одновременной ошибке запуска и очистки сообщаются обе; start_workers возвращает исходную ошибку запуска.

Код	Условие
0	Штатный конец гонки, предел раундов или справка
1	Ошибка аргументов, ввода-вывода, создания или освобождения ресурсов
130 / 143	SIGINT / SIGTERM при отсутствии другой ошибки

Неожиданная ошибка действующего mutex, sem_post, sem_wait или pthread_join приводит к сообщению и аварийному выходу с кодом 1. Полное присоединение работников в этом аварийном пути не гарантируется. Ожидаемые отказы запуска и destroy обрабатываются отдельно.

5 Интерфейс и варианты запуска

Сборка выполняется командой make. Необходимы Linux, GCC, Make и pthread; для проверок также Python 3. Основная программа собирается с C11, -Wall -Wextra -Wpedantic -Werror -O2 и -pthread.

```
make
./race
./race --help
```

Параметр	По умолчанию	Допустимое значение
--teams	3	1–8 команд
--cars	2	1–4 машины в команде
--lanes	4	1–6 дорожек
--length	24	3–120 клеток, больше числа машин
--laps	2	1–100 кругов
--rounds	100	1–10000 раундов
--seed	1	0–4294967295 в текущем Linux
--strategy	greedy	greedy или random
--delay	120	0–5000 мс

--log FILE сохраняет полный журнал, --quiet оставляет на экране параметры и итоги. --no-animation или --delay 0 включают обычный текстовый режим. --no-animation имеет приоритет над --delay при любом порядке аргументов. Числа проверяются полностью: отрицательные значения, переполнение, посторонние символы, неизвестные параметры и отсутствующие значения приводят к ошибке до запуска гонки.

Поклеточное отображение

Анимация включается при выводе в терминал и положительной задержке, если не заданы --quiet и --no-animation, а TERM отличается от dumb. Кадр обновляется после каждого перехода. По умолчанию пауза равна 120 мс; для медленного просмотра можно использовать ./race --delay 300.

На экране показаны раунд, очередь, маршрут, текущая машина, дорожка и клетка, круги, номер шага, потраченная стоимость и бюджет. В маршруте B означает «вперёд», И — «внутрь», Н — «наружу». Цвет обозначает команду, дополнительное выделение — текущую машину. Блокирование, нулевой ход и фактический сход имеют отдельные кадры.

Для полного кадра проверено окно не меньше 80×24. Длинная трасса показывается участком вокруг текущей машины; в меньшем окне подписи могут переноситься. При перенаправлении stdout анимация и паузы отключаются. Журнал не содержит ANSI-последовательностей и сохраняет события при --quiet.

```
./race --delay 300 --log race.log
./race --strategy random --seed 7 --no-animation
./race --quiet --rounds 1
```

Журнал включает стартовую расстановку, очередь, бюджеты, маршруты, переходы, смены дорожек, блокировки, нулевые ходы, круги, сход, финиш и итоговые места. Ошибки открытия, записи и закрытия учитываются в коде возврата.

6 Результаты контрольных запусков

Приведены результаты версии 2.0 в текущем Linux-окружении. --quiet не изменяет правила гонки. Фиксированный seed обеспечивает повторяемость в одном окружении; последовательность rand может отличаться между реализациями libc.

Стандартная гонка

```
./race --quiet
```

При 3 командах по 2 машины, 4 дорожках длиной 24 и 2 кругах все 6 машин финишировали за 11 раундов. Порядок машин: 2, 4, 5, 6, 1, 3. Продвижение каждой равно 48.

Место команды	Команда	Лучшая машина	Место машины
1	1	2	1
2	2	4	2
3	3	5	3

Остановка по пределу

```
./race --quiet --rounds 1
```

После одного раунда гонка завершилась по пределу. Порядок машин: 1, 5, 3, 6, 2, 4; продвижение соответственно 3, 3, 2, 2, 2, 1. Все получили состояние «не завершила». Порядок команд: 1, 3, 2. Нулевой код возврата означает штатную остановку по заданному ограничению.

Случайная стратегия

```
./race --quiet --strategy random --seed 7
```

Все машины финишировали за 24 раунда. Порядок машин: 6, 1, 4, 5, 2, 3; порядок команд: 3, 1, 2. Победила машина 6 команды 3, продвижение равно 48.

Плотная расстановка

```
./race --quiet --teams 8 --cars 4 --length 33 --lanes 2 \
  --laps 2 --strategy random --seed 0
```

Участвовали 32 машины. За 30 раундов финишировали 10, сошли 22. Первой стала машина 29 команды 8 с продвижением 66; затем машины 13 и 1. Первые три команды: 8, 4, 1. Сценарий проверяет заторы, нулевые ходы и удаление машин только после раунда.

Максимальные параметры

При 8 командах по 4 машины, 6 дорожках по 120 клеток, 100 кругах, пределе 10000 раундов и seed 4294967295 обе стратегии завершились с кодом 0: greedy за 1741 раунд, random за 4049. Проверены все 32 участника, команды, диапазоны мест и продвижения. Результаты получены в проверках CLI.

7 Инварианты и проверка модели

Проверки пересчитывают координаты, стоимость и занятость независимо от функций приложения. Для текстового режима состояние восстанавливается по журналу, для анимации кадры сопоставляются с восстановленным состоянием.

Инвариант	Как обеспечивается и проверяется
Не более одной машины в клетке	Маршрут исключает занятые клетки; переход обновляет track под mutex. Занятость независимо сверяется после шагов и в кадрах.
Нет движения назад и по диагонали	Допустимы только вперёд, внутрь и наружу. Тесты заново рассчитывают координаты переходов.
Стоимость не превышает бюджет	Шаг проверяется по остатку; наружу стоит 3. Тесты пересчитывают сумму и предел по фактической стоимости предыдущего хода.
Очередь обновляется после раунда	begin_round фиксирует порядок; end_round после уборки формирует следующий. Порядок сверяется с продвижением и дорожками.
Выбывшие больше не ходят	В очередь входят только ACTIVE. Сравниваются состояния Car и счётчики ходов потоков после схода или финиша.

Проверяются также запрет повторного посещения клетки, короткий финиш и сохранение препятствия до конца раунда. Уборка очищает клетку только при совпадении номера машины, поэтому не удаляет другого участника, занявшего её позднее.

Проверки правил и исполнителей

Набор	Проверенный объём
Малые трассы	3248 маршрутов с разными расстановками, бюджетами и обеими стратегиями
Сценарии модели на C	200 сценариев с проверкой маршрутов и занятости
Гонки с pthread	40 полных гонок, 11323 хода; pthread исполнителя соответствует своей машине и отличается от судьи
Независимые журналы CLI	28 журналов: очереди, бюджеты, круги, сход, финиш и обе классификации

Проверки выполнены в Linux с GCC и Python 3 командой make -B test. Полная пересборка и все сценарии завершились успешно, включая границы параметров, две максимальные конфигурации и ошибку записи stdout.

Сборка с предупреждениями как ошибками и статический анализ GCC -fanalyzer завершились без диагностик. В перечисленных сценариях нарушений пяти инвариантов не обнаружено.

8 Проверка синхронизации и интерфейса

Многопоточная версия сопоставлена с последовательной реализацией в Git. Проверки берут main.c из фиксированного коммита 3480fc4cf98ce7653480a155baa3fd36151f2a16, не переключая рабочую ветку.

Проверка	**Результат**
Сравнение с первой версией	Совпали 64 полных текстовых вывода
Изменение планирования	10 запусков с дополнительными задержками ожиданий сохранили вывод
Прерывание ожидания	33 искусственных EINTR не потеряли разрешений и подтверждений
Число потоков Linux	Через /proc наблюдались 33 одновременно существующих потока
Отказы запуска	Все 66 точек: mutex_init, 33 sem_init и 32 pthread_create; корректный откат
Отказы освобождения	34 точки destroy и 2 совмещённых отказа запуска и очистки; обработка остальных объектов продолжается
Завершение и сигналы	4 штатных сценария и 12 сценариев SIGINT/SIGTERM; join до destroy и закрытия журнала
Анимация	4976 кадров и 4010 переходов в четырёх сценариях; проверены сетка, координаты, маршрут, очередь, бюджет, события и цвета
Режимы вывода	Совпали журналы анимации, текста и --quiet; проверены ошибки файла и восстановление терминала

tests/faults.c собирается в отдельную библиотеку, которую тесты загружают через LD_PRELOAD. Обёртки подменяют выбранный вызов по счётчику и записывают события; обычные вызовы передают настоящим функциям через dlsym(RTLD_NEXT, ...). Например, отказ третьего pthread_create проверяет откат после создания двух работников. Это позволяет проверить все 66 точек запуска и порядок join, destroy и закрытия журнала.

Искусственный EINTR — возврат −1 с errno = EINTR из обёртки sem_wait без доставки сигнала. Он проверяет повтор ожидания. Реальные SIGINT/SIGTERM проверяются отдельно, с доставкой судье и работнику. Совпадение с первой версией показывает сохранение поведения; правильность правил дополнительно проверяется независимым восстановлением состояния.

Покадровая проверка охватывает окна 80×24 и 100×30, трассы длиной 3, 24, 33 и 120 клеток, до 32 машин и 6 дорожек. Сверяются все кадры, включая 22 схода в плотной гонке. Дополнительно проверены оба порядка --no-animation и --delay для задержек 1 и 5000 мс: кадры и паузы отсутствуют, текст совпадает.

```
make -B test
make test-display
make test-shutdown
```

ASan, UBSan и TSan недоступны в текущем окружении: пробная компоновка не прошла из-за отсутствующих runtime-библиотек. Проверка этими средствами не считается выполненной. Тесты подтверждают поведение в проверенных сценариях, но не гарантируют отсутствия всех ошибок и не заменяют динамическую проверку санитайзерами.

9 Использование ИИ и оценка решений

Я задавал требования к поведению гонки, поклеточной анимации и порядку доработок, обсуждал переход к версии 2.0 и передавал замечания к проекту. Codex использовался при подготовке модулей main.c, race.c, threads.c, display.c, тестов и документации. В частности, с его помощью реализованы передача ходов через семафоры и проверки отказов. Ниже приведены два конкретных исправления и способы их проверки.

Приоритет отключения анимации

В исходном варианте --no-animation устанавливал delay_ms в 0. Если следом шёл --delay 1, задержка снова становилась положительной и анимация включалась. Это противоречило смыслу флага. Ошибка была указана в замечаниях к проекту; добавленный терминальный тест воспроизвёл её до исправления.

В Config добавлен отдельный no_animation. Разбор --delay теперь меняет только задержку, а запуск анимации отдельно проверяет запрет. После изменения тест проверяет оба порядка аргументов с задержками 1 и 5000 мс: управляющих последовательностей нет, полный текст совпадает с обычным режимом, ожидания кадров отсутствуют.

Продолжение очистки после ошибки

В раннем варианте ошибка sem_destroy или pthread_mutex_destroy вызывала немедленный exit. Оставшиеся объекты не проходили очистку; при откате могла теряться первичная причина неудачного запуска.

stop_workers сохраняет первую ошибку и продолжает освобождение остальных объектов после join. Флаг ресурса сбрасывается только при успешном destroy. start_workers возвращает исходную ошибку запуска, а ошибку очистки сообщает отдельно. Проверка через LD_PRELOAD охватывает 34 точки отказа destroy и два совмещённых отказа запуска и очистки. Журнал операций подтверждает попытки обработать остальные объекты и закрыть файл после присоединения работников.

Оценка архитектуры

По правилам бюджет и свободные клетки следующей машины зависят от предыдущего хода. Поэтому одновременно существуют отдельные pthread водителей, но движения упорядочены. Семафоры связывают судью с конкретным исполнителем; mutex защищает общий ход, включая вывод и паузы. Цель схемы — моделирование взаимодействия участников, а не ускорение гонки.

Трасса и стратегии имеют границы, описанные в разделах 1 и 2. Вывод использует stdio; отдельный ввод-вывод через open/read/write и вторая программа с условными переменными не реализованы. Это дополнительные требования.

Источники и материалы проекта

1. i-tasks.pdf. Вариант 20 «Автогонки», страницы 40–42.
2. i-tasks-req-02.pdf. Требования к ИДЗ 2, страницы 2 и 5–7.
3. Исходники main.c, race.c, threads.c, display.c и заголовки версии 2.0.
4. AUDIT.md — сверка требований и проверки от 24 октября 2026 года; ARCHITECTURE.md — устройство программы; README.md — инструкция запуска.